PASSWORD SECURITY & HASHING LAB REPORT

Internship Week 1 Task: Password Security and Hashing Lab

Student: Muhammad Haseeb
Program: BS Cybersecurity, IMSciences Peshawar
Platform: TryHackMe: Crack the Hash Room
Tools: CrackStation, Hashes.com, Hashcat, and hashid
Difficulty: Intermediate
Submission: Internship Group Leader, Week 1

This report documents the theoretical concepts studied, the practical lab results, the tools and methodology used, and the key lessons learned during the Password Security and Hashing Lab.

1. OBJECTIVE

This lab was conducted as part of Internship Week 1 and focused on password security fundamentals. The objective was to understand how passwords are stored, why weak password storage can be exploited, and how user credentials can be protected in real world applications. The lab combined theoretical study with authorized hands on practice using TryHackMe's Crack the Hash room.

The lab objectives were to:

- Understand how passwords are stored using different techniques and algorithms.
- Identify weaknesses in each storage method through practical analysis.
- Practice hash identification and password cracking only on authorized, lab provided hashes.
- Recommend secure password storage practices for SafeX clients.
- Document the tools, commands, results, and lessons in a professional report.

2. CONCEPTS STUDIED

2.1 Plaintext Storage

In plaintext storage, passwords are saved exactly as the user enters them, with no security layer applied. For example, if a user chooses the password **Pakistan123**, that exact value is stored in the database.

This is the most dangerous storage method. If an attacker gains access to the database, every password is immediately readable and no cracking is required. Exposed passwords may also be reused in credential stuffing attacks because many users use the same password across multiple services.

2.2 Encryption

Encryption converts readable plaintext into ciphertext by using an algorithm and a secret key. It is reversible because the original value can be recovered with the correct decryption key.

Why encryption is unsuitable for password storage: A system should verify a password, not recover it. If an attacker obtains both the database and the decryption key, every stored password can be decrypted. Passwords should therefore be protected with a dedicated password hashing function rather than reversible encryption.

2.3 Hashing

A hash function converts an input into a fixed length output called a hash value. Password verification works by hashing the password submitted during login and comparing the result with the stored password hash. A secure password hashing scheme is designed to be one way: the original password is not recovered from the stored hash.

Attackers generally do not reverse a hash. Instead, they guess candidate passwords, process each guess with the same hashing method, and compare the results. If the values match, the candidate password has been found.

2.4 Salting

A salt is a unique, cryptographically random value generated for a password before it is processed by the password hashing function. Unique salts ensure that identical passwords produce different stored values and prevent attackers from reusing the same precomputed work across many accounts.

- Use a unique salt for every password record.
- Store the salt with the password hash; the salt does not need to be secret.
- Salts reduce the value of rainbow tables and large precomputed lookup databases.
- Salting does not make a weak password strong and does not replace a strong password policy.

2.5 Key Stretching

Key stretching increases the computational cost of password verification. Password hashing algorithms such as bcrypt, PBKDF2, scrypt, and Argon2id are intentionally configured to make each password guess more expensive. This has a manageable effect on legitimate logins while increasing the cost of large scale password guessing.

2.6 Slow Hashing

Slow password hashing uses algorithms that are deliberately expensive to compute. Fast general purpose hashes such as MD5, SHA 1, and plain SHA 256 were designed for speed and are not appropriate for storing passwords. Password specific algorithms such as Argon2id, bcrypt, scrypt, and PBKDF2 are designed to make repeated guessing more costly.

2.7 Memory Hard Hashing

Memory hard password hashing requires substantial memory as well as processing power. This reduces the advantage of highly parallel hardware because each concurrent guess consumes memory. Argon2id is a modern memory hard password hashing algorithm and is widely recommended for new systems when it is configured appropriately for the application's environment.

3. PASSWORD ATTACK METHODS

Understanding common password attacks is essential for designing effective defenses. The following methods were studied in the context of authorized security testing and defensive password design.

3.1 Brute Force Attack

Risk level: High against short or weak passwords
How it works: Systematically tests possible character combinations until a match is found. The search space grows rapidly as password length and unpredictability increase.
Defense: Use long, unique passwords or passphrases and enable MFA. Each additional unpredictable character increases the search space.
Common tools: Hashcat and John the Ripper

3.2 Dictionary Attack

Risk level: High against common passwords
How it works: Tests words and known password candidates from a wordlist such as rockyou.txt. It is effective because many passwords are based on common words, names, and predictable patterns.
Defense: Use long, unique, randomly generated passwords or strong passphrases. Block commonly breached passwords.
Common tools: Hashcat and John the Ripper

3.3 Rainbow Table Attack

Risk level: High against unsalted fast hashes
How it works: Uses precomputed mappings between candidate passwords and hash values. This can make lookups very fast when the target hashes are unsalted and use a fast algorithm.
Defense: Use a modern password hashing algorithm with a unique salt for every password.
Common tools: RainbowCrack and precomputed tables

3.4 Credential Stuffing

Risk level: High where passwords are reused
How it works: Reuses username and password pairs exposed in previous breaches. Automated tools test those credentials against other services.
Defense: Require unique passwords, check passwords against breach data, apply rate limits, and enable MFA.
Common tools: Automated credential testing tools

3.5 Mask Attack

Risk level: Moderate when the password pattern is known
How it works: Uses known password structure, such as a fixed length or a predictable suffix, to reduce the number of candidates that must be tested.
Defense: Avoid predictable formats such as names followed by dates or numbers. Use randomly generated passwords.
Common tools: Hashcat mask mode

3.6 Phishing and Social Engineering

Risk level: High because cryptography may be bypassed
How it works: Tricks users into entering credentials on a fake site or disclosing them through deceptive communication. The attacker obtains the password directly rather than cracking a hash.
Defense: Verify domains, avoid untrusted login links, use phishing resistant MFA where possible, and provide security awareness training.

Common tools: GoPhish for authorized security testing

4. TOOLS USED

All hash identification was performed before cracking attempts. The workflow was to identify the likely hash type, select the appropriate method, and test only the hashes provided in the authorized lab.

Tool	Purpose
TryHackMe	Authorized lab environment that provided the Crack the Hash challenges.
hashid	Identified likely hash formats before cracking attempts.
Hashcat	Used for hash mode reference and authorized dictionary based cracking.
CrackStation	Used as an online lookup service for common, unsalted hash values.
Hashes.com	Used as a secondary lookup service for selected lab hashes.

Command Reference

```
hashid '<hash>'
hashid -m '<hash>'
hashcat -m <mode> '<hash>' /usr/share/wordlists/rockyou.txt
hashcat -m <mode> '<hash>' --show
```

Tool selection rationale: online lookup services were used for quick checks of common lab hashes, while Hashcat provided a local command line option for authorized dictionary attacks. The correct tool depends on the hash type, the available evidence, and the rules of engagement.

5. METHODOLOGY

- Receive the hash from the authorized TryHackMe challenge.
- Run hashid to identify likely hash formats.
- Use hashid -m to review possible Hashcat mode numbers.
- Check an appropriate authorized lookup service for common hashes.
- If necessary, run a controlled Hashcat dictionary attack using an approved wordlist.
- Record the hash type, tool used, result, and lesson learned.

6. LAB RESULTS

All hashes in this section were provided exclusively by TryHackMe's Crack the Hash room. No real world credentials were used or tested.

6.1 Beginner Level Results

No.	Hash Type	Tool Used	Password Found	Status
1	MD5	CrackStation	easy	Cracked
2	SHA 1	CrackStation	password123	Cracked
3	SHA 256	CrackStation	letmein	Cracked
4	bcrypt	Hashes.com	bleh	Cracked
5	MD4	CrackStation	Eternity22	Cracked

6.2 Intermediate Level Results

No.	Hash Type	Tool Used	Password Found	Status
1	SHA 256	CrackStation	paule	Cracked
2	NTLM	CrackStation	n63umy8lkf4i	Cracked
3	SHA 512crypt	Hashes.com	waka99	Cracked
4	SHA 1	Hashes.com	481616481616	Cracked

6.3 Overall Summary

Metric	Result
Total hashes attempted	9
Total hashes cracked	9 (100% success rate)
Beginner level	5 of 5 cracked
Intermediate level	4 of 4 cracked
Hash types encountered	MD5, MD4, SHA 1, SHA 256, SHA 512crypt, bcrypt, and NTLM
Primary identification method	hashid and hashid -m
Primary lookup tool	CrackStation
Secondary lookup tool	Hashes.com

7. KEY LEARNINGS

Fast hashes are unsuitable for password storage

MD5, SHA 1, and other fast general purpose hashes can be evaluated at very high speed. They do not provide adequate resistance to modern password guessing and should not be used as password storage functions.

Password strength and password hashing work together

A strong password hashing algorithm cannot compensate for a trivial password. The bcrypt example in this lab was cracked because the underlying password was extremely weak.

Correct identification improves the workflow

Identifying the likely hash type before selecting a cracking method reduces errors and helps analysts choose an appropriate tool or Hashcat mode.

Precomputed lookups are most effective against weak, unsalted hashes

Large lookup databases can quickly recover common passwords when the target uses an unsalted, fast hash. Unique salts make precomputation far less reusable.

Modern defenses are layered

Secure password storage requires a modern password hashing algorithm, appropriate parameters, unique salts, strong password controls, rate limiting, and MFA.

8. CONCLUSION

This lab demonstrated why weak password storage is a serious application security risk. All nine authorized lab hashes were recovered using publicly available tools and services, showing how quickly weak or common passwords can be exposed when unsuitable storage methods are used.

The main conclusions are:

- Do not use fast general purpose hashes such as MD5, SHA 1, or plain SHA 256 for password storage.
- Use a dedicated password hashing algorithm such as Argon2id, bcrypt, scrypt, or PBKDF2 with appropriate parameters.
- Use unique salts and enforce strong, unique passwords.
- Apply rate limiting and MFA as additional layers of protection.
- Security professionals must understand attack methods in order to design effective defenses.